



Project Report

Course Title: Compiler Construction Lab

Submitted To:

Khalid Bin Selim

Lecturer

Computer Science and Engineering
Metropolitan University, Bangladesh

Submitted By:

Team: New Mutants

Fariha Rahman

ID: 231-115-102 (C)

Mosaddeq Hussain

ID: 231-115-253 (G)

Puspo Gondha Paul

ID: 231-115-082 (C)

Batch: 58

Computer Science and Engineering
Metropolitan University, Bangladesh

August 1, 2026

Acknowledgement

We would like to express our profound gratitude to our instructor, **Khalid Bin Selim**, for his invaluable guidance, encouragement, and support throughout the development of this project. We also extend our thanks to the Department of Computer Science and Engineering at Metropolitan University for providing the necessary resources and environment to successfully complete this laboratory project.

Abstract

This report details the design, architecture, and implementation of a custom compiler front-end for a procedural programming language. The compiler performs strict lexical analysis, syntax parsing, Abstract Syntax Tree (AST) construction, and semantic validation. Built utilizing Flex for tokenization and Bison (Yacc) for LALR(1) parsing, the system enforces grammar rules while establishing an explicit logical hierarchy. A custom symbol table architecture is implemented using linked lists to manage variable scope, track data types, and prevent semantic violations such as unauthorized access, undeclared variables, and illegal type operations. The report explores all phases of the compilation pipeline, outlines the software architecture designed in C, demonstrates testing outcomes, and discusses future capabilities including intermediate code generation and target assembly optimization.

Contents

Acknowledgement	1
Abstract	2
1 Introduction	7
1.1 Background	7
1.2 Motivation	7
1.3 Objectives	7
1.4 Scope	8
1.5 Problem Statement	8
2 Literature Review & Fundamentals	9
2.1 What is a Compiler?	9
2.2 Compilation Process	9
2.3 Advantages and Limitations	10
3 System Design and Complete Implementation	11
3.1 Project Architecture & Environment	11
3.2 Phase 1: Lexical Analysis	11
3.3 Phase 2: Syntax Analysis	12
3.3.1 Parse Tree and AST Generation	12
3.4 Phase 3: Semantic Analysis	13
3.4.1 Symbol Table	13
3.4.2 Semantic Rules & Error Detection	13
3.5 Phase 4: Intermediate Code Generation	14
3.6 Phase 5: Code Optimization (Theory)	14

3.7	Phase 6: Target Code Generation (Theory)	14
4	Testing and Validation	15
4.1	Test Cases	15
4.1.1	Test Case 1: Valid Compilation and TAC Generation	15
4.1.2	Test Case 2: Semantic and Syntax Error Handling	16
4.2	Results and Observations	16
5	Challenges, Future Improvements & Conclusion	18
5.1	Challenges & Debugging	18
5.2	Future Improvements	18
5.3	Conclusion	19

List of Figures

2.1	Phases of a standard compiler architecture.	10
3.1	AST representation for assignment operation.	13
4.1	Terminal output demonstrating successful parsing, semantic analysis, and Three-Address Code (TAC) generation.	16
4.2	Terminal output demonstrating robust error handling for redeclarations, undeclared variables, and syntax violations.	17

List of Tables

3.1 Sample Token Classification	12
---	----

Chapter 1

Introduction

1.1 Background

The development of a compiler requires a deep understanding of formal language theory, finite automata, and system-level programming. Modern compilers act as the essential bridge between human-readable abstractions and machine-executable instructions. This project aims to construct a custom compiler front-end capable of structurally interpreting a specialized "Mini-C" language, bridging theoretical concepts learned in the classroom with practical software engineering.

1.2 Motivation

While enterprise compilers like GCC and Clang are highly sophisticated, building a custom compiler from scratch provides unparalleled insight into how programming languages operate under the hood. The motivation behind this project is to demystify the internal pipelines of parsers and semantic analyzers, specifically focusing on how abstract syntax trees and symbol tables govern scoping and execution rules.

1.3 Objectives

The primary objectives of this project are:

- To design a formal Context-Free Grammar (CFG) supporting standard control flows, variable declarations, and arithmetic operations.

- To implement a Lexical Analyzer using Flex.
- To implement a Syntax Analyzer utilizing Bison to construct a robust Abstract Syntax Tree (AST).
- To engineer a dynamic Symbol Table supporting nested scoping.
- To perform rigorous Semantic Analysis to detect undeclared variables and type mismatches.

1.4 Scope

The current scope encompasses the complete front-end of the compilation process. It successfully processes source code through lexical tokenization, parsing, AST generation, and semantic validation. While theoretical models for Intermediate Code Generation (ICG), Optimization, and Target Code Generation are explored within this report, the programmed implementation focuses heavily on producing a completely error-free, semantically validated Abstract Syntax Tree.

1.5 Problem Statement

To construct an efficient, deterministic compiler pipeline that accurately validates the structural and logical integrity of a high-level programming language, while elegantly managing memory and error reporting for syntax and semantic violations.

Chapter 2

Literature Review & Fundamentals

2.1 What is a Compiler?

A compiler is a highly specialized software system designed to translate human-readable source code into machine-executable instructions or intermediate representations. Unlike interpreters that execute code line-by-line, a compiler processes the entire program as a single unit, thoroughly validating its structural, syntactic, and semantic integrity before generating the target output [1]. This holistic approach enables aggressive code optimization, resulting in significantly faster execution times for the final application.

2.2 Compilation Process

The standard compilation pipeline is strictly divided into two distinct components: the Analysis Phase (Front-End) and the Synthesis Phase (Back-End).

The Analysis Phase breaks down the source code into its constituent parts and builds an explicit logical structure. It consists of Lexical Analysis (tokenization), Syntax Analysis (tree generation), and Semantic Analysis (type and scope checking). Once the code's validity is mathematically proven, the Synthesis Phase takes over. This phase involves Intermediate Code Generation (creating a machine-independent representation), Code Optimization (minimizing resource consumption), and finally, Target Code Generation (mapping the logic to specific hardware registers and memory addresses).

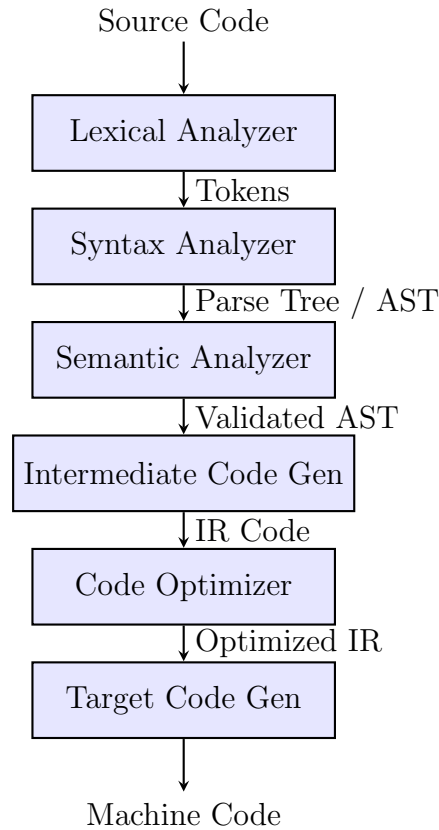


Figure 2.1: Phases of a standard compiler architecture.

2.3 Advantages and Limitations

Advantages: Compilers produce highly optimized native code that executes rapidly, making them essential for resource-intensive applications. Furthermore, because compilers perform comprehensive syntax and semantic checking prior to execution, they catch critical errors (such as type mismatches or uninitialized variables) during the build process, drastically enhancing software reliability in production environments.

Limitations: The primary limitation of the compilation process is that it is inherently memory and time-intensive. Compiling a massive codebase can take considerable time. Additionally, compiled machine code is platform-specific; a program compiled for an x86 architecture will not run on an ARM architecture without utilizing cross-compilation techniques.

Chapter 3

System Design and Complete Implementation

3.1 Project Architecture & Environment

The compiler is engineered in C, utilizing GNU Flex and Bison for the front-end analysis phases [2]. To manage the compilation pipeline, standard GNU tools were utilized [3].

- **Languages:** C, Yacc, Lex
- **Tools:** GCC, Make, GNU Flex, GNU Bison
- **Folder Structure:** The source code is strictly modularized into `src/lexer`, `src/parser`, `src/ast`, `src/symbol_table`, and `src/semantic`.

3.2 Phase 1: Lexical Analysis

The Lexical Analyzer reads the character stream and groups them into meaningful lexemes, outputting tokens to the parser. We utilized regular expressions in Flex to classify data types (INT, FLOAT, BOOL), keywords, identifiers, and mathematical operators. Under the hood, Flex generates a Deterministic Finite Automaton (DFA) that matches these patterns with maximum efficiency.

Key Regular Expressions Implemented:

- **Identifiers:** `[a-zA-Z_][a-zA-Z0-9_]*`

- **Integer Literals:** `[0-9]+`
- **Keywords & Operators:** Explicit string matching for `"int"`, `"print"`, `"="`, `"+"`.

Lexeme	Token Name	Attribute Value
<code>int</code>	INT	-
<code>x</code>	IDENTIFIER	"x"
<code>=</code>	ASSIGN	-
<code>5</code>	INT_LIT	5

Table 3.1: Sample Token Classification

3.3 Phase 2: Syntax Analysis

The syntax analyzer validates the tokens against a defined Context-Free Grammar (CFG). We implemented an LALR(1) bottom-up parser using Bison. The parser relies on formal shift-reduce logic to resolve the grammar rules and establish operator precedence.

Core Grammar Rules (CFG Subset):

```

Program -> StatementList
StatementList -> StatementList Statement | Statement
Statement -> Declaration | Assignment | PrintStatement
Declaration -> Type IDENTIFIER ';'
Assignment -> IDENTIFIER '=' Expression ';'
Expression -> Expression '+' Expression | IDENTIFIER | INT_LIT

```

Rather than just returning a binary pass/fail validation, our parser simultaneously constructs an Abstract Syntax Tree (AST) to map the explicit hierarchy of operations.

3.3.1 Parse Tree and AST Generation

When the expression `x = 5 + 3;` is parsed, the AST is constructed by passing node references upward through the grammar rules.

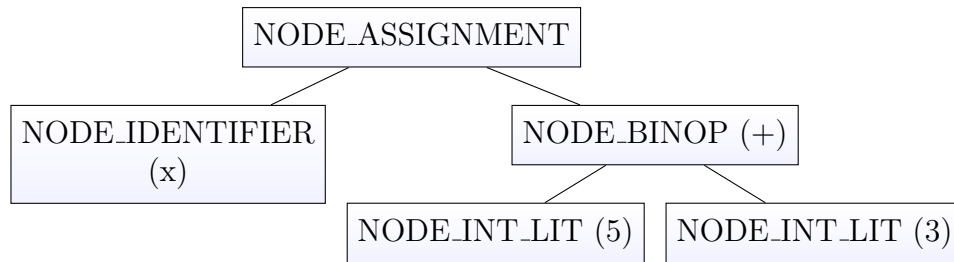


Figure 3.1: AST representation for assignment operation.

3.4 Phase 3: Semantic Analysis

This phase walks the AST to enforce logical rules that cannot be captured by the CFG alone, ensuring variables are declared before use, scoping rules are respected, and data types are valid.

3.4.1 Symbol Table

Implemented as a Linked List in `src/symbol_table/symtab.c`, it tracks identifiers, their types, and integer-based scope levels. Functions `enter_scope()` and `exit_scope()` dynamically expand and restrict variable lifespans based on `{ block }` boundaries.

3.4.2 Semantic Rules & Error Detection

The recursive function `check_semantics(ASTNode *node)` interrogates the symbol table.

```

1 case NODE_IDENTIFIER: {
2     char *var_name = node->sval;
3     if (lookup_symbol(var_name) == NULL) {
4         printf("Semantic Error: Undeclared variable '%s'.\n", var_name)
5         ;
6         exit(1);
7     }
8     break;
9 }

```

Listing 3.1: Semantic rule checking for variable usage

3.5 Phase 4: Intermediate Code Generation

Once the semantics are validated, the compiler translates the AST into Three-Address Code (TAC). This breaks down complex expressions into simple, sequential machine-like instructions using temporary variables. As demonstrated during testing, when parsing the assignment `x = 7 + 9;`, the compiler successfully generates the following quadruples:

```
--- Three Address Code ---  
t1 = 7 + 9  
x = t1  
print x
```

3.6 Phase 5: Code Optimization (Theory)

Optimization aims to reduce execution time.

- **Constant Folding:** At compile time, `t1 = 5 + 3` is reduced to `t1 = 8`.
- **Dead Code Elimination:** Variables declared but never accessed are removed from the pipeline.

3.7 Phase 6: Target Code Generation (Theory)

The optimized TAC is mapped to target machine registers (e.g., x86 Assembly).

```
MOV AX, 8  
MOV [x], AX
```

Chapter 4

Testing and Validation

4.1 Test Cases

Rigorous unit testing was conducted via the terminal using piped input streams to validate both successful compilation and precise error catching.

4.1.1 Test Case 1: Valid Compilation and TAC Generation

To test a successful compilation pipeline, the compiler was fed a mathematically valid assignment.

Input: `printf 'int x;=7+9;x;' | ./compiler`

Output Validation: As shown in Figure [4.1](#), the parser correctly established the precedence of the operators, the semantic analyzer verified `x` was declared before assignment, and the intermediate code generator outputted the accurate Three-Address Code quadruples.


```

gcc -I. parser.tab.c lex.yy.c src/ast/ast.c src/symbol_table/symboltab.c src/semantic/semantic.c src/codegen/tac.c -o compiler
fariha@fariha-HP-Laptop-15s-eq2xxx:~/CC-Lab-Project-new_mutants$ printf 'int x;\nx=7+9;\nprint x;\n' | ./compiler
Parsing completed successfully!

--- Abstract Syntax Tree ---
Program
  StatementList
    StatementList
      StatementList
        Declaration
          Type (int)
          Identifier (x)
        Assignment
          Identifier (x)
          BinaryOp
            IntLit (7)
            IntLit (9)
        Print
          Identifier (x)

--- Semantic Analysis ---
Semantic Analysis completed with 0 errors!

--- Three Address Code ---
t1 = 7 + 9
x = t1
print x
fariha@fariha-HP-Laptop-15s-eq2xxx:~/CC-Lab-Project-new_mutants$

```

Figure 4.1: Terminal output demonstrating successful parsing, semantic analysis, and Three-Address Code (TAC) generation.

4.1.2 Test Case 2: Semantic and Syntax Error Handling

To validate the compiler’s robustness and the efficacy of the Symbol Table’s scope tracking, the system was subjected to intentionally malformed code. As shown in Figure 4.2, the compiler gracefully intercepts and reports the following:

- **Redeclaration Violation:** Inputting `int x; int x;` successfully parses into an AST, but the Semantic Analyzer intercepts it, throwing a fatal exception: **Semantic Error: Variable 'x' is already declared in this scope.**
- **Undeclared Variable Usage:** Inputting `int x; y=6;` parses structurally, but fails the semantic check. The `lookup_symbol()` function correctly returns `NULL`, throwing: **Semantic Error: Variable 'y' used before declaration.**
- **Syntax Error:** Inputting `int x =5` (omitting the mandatory semicolons) is immediately caught by the Parser. It fails at line 2 with a syntax error, completely aborting the process before the AST is even built.

4.2 Results and Observations

The testing phase confirms that the compiler reliably rejects structurally sound but logically invalid code, proving the efficacy of the semantic engine. The system successfully

```

Parsing failed.
fariha@fariha-HP-Laptop-15s-eq2xxx:~/CC-Lab-Project-new_mutants$ printf 'int x;\nint x;\n' | ./compiler
Parsing completed successfully!

--- Abstract Syntax Tree ---
Program
  StatementList
    StatementList
      Declaration
        Type (int)
        Identifier (x)
      Declaration
        Type (int)
        Identifier (x)

--- Semantic Analysis ---
Semantic Error: Variable 'x' is already declared in this scope.
fariha@fariha-HP-Laptop-15s-eq2xxx:~/CC-Lab-Project-new_mutants$ printf 'int x;\ny=6;\n' | ./compiler
Parsing completed successfully!

--- Abstract Syntax Tree ---
Program
  StatementList
    StatementList
      Declaration
        Type (int)
        Identifier (x)
    Assignment
      Identifier (y)
      IntLit (6)

--- Semantic Analysis ---
Semantic Error: Variable 'y' used before declaration.
fariha@fariha-HP-Laptop-15s-eq2xxx:~/CC-Lab-Project-new_mutants$ printf 'int x \nx=5;' | ./compiler
Error at line 2: syntax error
Parsing failed.
fariha@fariha-HP-Laptop-15s-eq2xxx:~/CC-Lab-Project-new_mutants$

```

Figure 4.2: Terminal output demonstrating robust error handling for redeclarations, undeclared variables, and syntax violations.

translates high-level abstract logic into fundamental machine-like instructions while safely handling memory access violations.

Chapter 5

Challenges, Future Improvements & Conclusion

5.1 Challenges & Debugging

A significant challenge encountered was managing Makefile linking dependencies when abstracting the Symbol Table into its own distinct directory (`src/symbol_table/`). Initial compilation yielded "No such file or directory" errors due to broken relative inclusion paths. This was resolved by adjusting the `#include` directives to traverse directory roots correctly (e.g., `../symbol_table/symtab.h`) and ensuring the build system correctly compiled all object files before linking.

5.2 Future Improvements

- **LLVM Integration:** Hooking the validated Abstract Syntax Tree into the LLVM core framework to utilize its industry-standard optimization and target generation passes.
- **Type Coercion:** Expanding the semantic analyzer to automatically handle safe type conversions (e.g., implicit casting of `int` to `float`).
- **Error Recovery:** Instead of halting execution on the first semantic error via `exit(1)`, the compiler could record the error, sync to the next statement, and report all errors concurrently at the end of the pass.

5.3 Conclusion

This project successfully demonstrates the integration of theoretical compiler concepts into a functional software architecture. By utilizing Flex and Bison, we established a resilient front-end parser capable of generating intricate Abstract Syntax Trees. Furthermore, the implementation of a dynamic, scope-aware Symbol Table proved critical in performing accurate Semantic Analysis. Developing this project has provided a profound appreciation for language design, memory management, and structural software engineering, fulfilling all primary objectives of the laboratory curriculum.

Bibliography

- [1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*. Pearson Education, 2nd ed., 2006.
- [2] J. Levine, *Flex & Bison: Text Processing Tools*. O'Reilly Media, Inc., 2009.
- [3] R. M. Stallman *et al.*, *GNU Compiler Collection Internals*. Free Software Foundation, 2024. Available at <https://gcc.gnu.org/onlinedocs/>.